Manual de Instalación y Uso

API de Integración para POS

Versión 2.0

1. Funcionalidad

Este servicio actúa como un puente de comunicación entre los sistemas de venta (cajas registradoras, aplicaciones de cobro, etc.) y los terminales POS (puntos de venta físicos).

Permite:

- Ejecutar pagos con tarjeta (chip, contactless), códigos QR, QR Pix y pagos con criptomonedas (Crypto).
- Anular transacciones previamente realizadas (solo para pagos con tarjeta).
- Realizar cierres de lote en los terminales POS.
- Consultar el estado de los dispositivos conectados en tiempo real.

El servicio expone una API REST que recibe peticiones HTTP y las traduce al protocolo que entienden los POS, devolviendo una respuesta normalizada.

2. Requisitos previos

Antes de instalar, asegure que su entorno cumple con:

Requisito	Detalle
Sistema operativo	Linux (recomendado), Windows Server o macOS (para pruebas).
Node.js	Versión 16 o superior. Verifique con node -v.
npm	Versión 6 o superior. Se instala junto con Node.js.
PM2	(Opcional pero recomendado) Gestor de procesos. Instalar con npm install -g pm2.
Acceso de red	Los POS deben poder conectarse al servidor por el puerto TCP 5455. El servidor debe ser accesible desde los POS.
Puertos disponibles	TCP 5455 para comunicación con POS, y HTTP 7392 para la API (modificables vía variables de entorno).
IP fija para los POS	Cada terminal POS debe tener una dirección IP fija o estar en la misma subred que el servidor.

3. Pasos de instalación

3.1 Descargar el paquete

Descomprima el archivo entregado (integracion-pos.zip o del repositorio proporcionado de github) en el directorio donde desea alojar el servicio.

```
unzip integracion-pos.zip -d /opt/integracion-pos
cd /opt/integracion-pos
```

3.2 Instalar dependencias

Ejecute:

```
npm install
```

Esto instalará las dependencias necesarias.

3.3 Configurar dispositivos (devices.json)

Cree o edite el archivo devices.json en la raíz del proyecto. Este archivo mapea los nombres (etiquetas) de cada caja a su dirección IP real.

Estructura:

```
{
  "Caja1": "192.168.1.100",
  "Caja2": "192.168.1.101",
  "Caja3": "10.0.0.50"
}
```

- Clave: Nombre público que usará en las peticiones (ej. Caja1).
- Valor: Dirección IP del terminal POS.

Importante: La IP debe coincidir exactamente con la dirección desde la que el POS se conecta al servidor. Si el POS está detrás de NAT, use la IP que el servidor ve en la conexión entrante.

3.4 Ajustar puertos mediante variables de entorno

Puede cambiar los valores por defecto de los puertos de la API y POS, mediante variables de entornos, para ello crear un archivo en la raíz del proyecto llamado .env y configurar las siguientes variables:

```
POS_PORT=5455   # Puerto TCP para POS
API_PORT=7392   # Puerto HTTP para la API
```

O bien, definirlos al iniciar:

```
POS_PORT=5455 API_PORT=7392 node index.js
```

3.5 Iniciar el servicio

Para probar el servicio de forma interactiva:

```
node index.js
```

Debería ver en consola:

- Los dispositivos configurados (tabla con nombres e IPs).
- Mensaje: Servidor TCP iniciado en el puerto 5455.
- Mensaje: Servidor web iniciado en el puerto 7392.

Para mantenerlo en producción, use PM2 (ver sección 4).

4. Uso con PM2 y logrotate

PM2 mantiene el proceso activo y lo reinicia automáticamente en caso de fallo.

4.1 Instalar PM2 (si no lo tiene)

```
npm install -g pm2
```

4.2 Iniciar el servicio con PM2

```
pm2 start index.js --name integracion-pos
```

4.3 Configurar inicio automático al arrancar el sistema

```
pm2 startup
pm2 save
```

4.4 Configurar logrotate para PM2

Los logs de PM2 pueden crecer indefinidamente. Para rotarlos automáticamente:

a) Instalar pm2-logrotate (módulo oficial):

```
pm2 install pm2-logrotate
```

b) Configurar el tamaño máximo y retención:

```
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
pm2 set pm2-logrotate:compress true
pm2 set pm2-logrotate:dateFormat YYYY-MM-DD_HH-mm-ss
```

- max_size: 10 MB por archivo de log.
- retain: conservar los últimos 7 archivos rotados.
- compress: comprimir los logs antiguos.

c) Verificar configuración:

```
pm2 conf pm2-logrotate
```

4.5 Comandos útiles de PM2

Comando	Descripción
pm2 status	Ver estado del servicio.
pm2 logs integracion-pos	Ver logs en tiempo real.
pm2 restart integracion-pos	Reiniciar el servicio.
pm2 stop integracion-pos	Detener el servicio.
pm2 delete integracion-pos	Eliminar el servicio de PM2.

5. Endpoints de la API

Todos los endpoints requieren POST (excepto /dispositivos que es GET) y envían y reciben JSON.

5.1 Pago (chip, contactless, QR, QR Pix o Crypto)

POST /pago/

Cuerpo general:

```
{
  "device": "Caja1",
  "typePay": "chip",
  "importe": 150.50,
  "crypto": {         // Solo para typePay = "crypto"
    "network": "Polygon",
    "paymentKey": "004"
  },
  "pix": {            // Solo para typePay = "qrpix"
    "cpf": "12345678901",
    "phone": "551199999999"  // Opcional
  }
}
```

Campos comunes (obligatorios para todos los métodos)

Campo	Tipo	Descripción
device	string	Etiqueta definida en devices.json (o IP directa).
typePay	string	Método de pago: "chip", "ctl" (contactless), "qr", "qrpix" o "crypto".
importe	number/string	Monto con dos decimales (ej. 150.50 o "150.50").

Campos específicos por método

Método (typePay)	Campos adicionales	Obligatorio	Descripción
chip	-	-	Pago con tarjeta de crédito/débito (chip).

Método (typePay)	Campos adicionales	Obligatorio	Descripción
ctl (contactless)	-	-	Pago sin contacto (tarjeta o dispositivo móvil).
qr	-	-	Pago mediante código QR genérico.
qrpix	pix.cpf	Sí	CPF del pagador (11 dígitos).
qrpix	pix.phone	No	Teléfono del pagador (1-12 dígitos).
crypto	crypto.network	Sí	Red de criptomonedas (ej. "Polygon").
crypto	crypto.paymentKey	Sí	Clave de pago asignada (3 dígitos, como string para conservar ceros).

Nota: Para typePay = "crypto", el campo paymentKey debe enviarse como string (ej. "004") para preservar los ceros a la izquierda.

Respuesta exitosa (ejemplo):

```
{
  "status": "success",
  "message": "TRANS. APROBADA",
  "data": [
    { "name": "authCode", "value": "123456" },
    { "name": "purchaseAmount", "value": "0000000015050" },
    { "name": "additionalAmount", "value": "0000000000500" }  // Si aplica propina/donación
  ]
}
```

Posibles estados en status: success o error.

5.2 Anulación de venta

POST /anular/

Cuerpo:

```
{
  "device": "Caja1",
  "reference": 12345
}
```

reference: número de referencia de la transacción a anular (6 dígitos, puede tener ceros a la izquierda; se envía como número).

Nota: Solo se pueden anular operaciones realizadas con chip o contactless. Los pagos QR, QR Pix y Crypto no son anulables mediante este endpoint.

5.3 Cierre de lote

POST /cerrar/

Cuerpo:

```
{
```

```
  "device": "Caja1"
}
```

Respuesta:

```
{
  "status": "success",
  "message": "Cierre de lote ejecutado correctamente",
  "data": [ ... ]   // Lista de transacciones del lote (si las hay)
}
```

5.4 Estado de dispositivos

GET /dispositivos

Devuelve el estado en tiempo real de todos los POS configurados y conectados.

Respuesta ejemplo:

```
{
  "actualizado": "2026-08-31T15:30:00-04:00",
  "total": 2,
  "conexionesTCP": 2,
  "dispositivos": [
    {
      "name": "Caja1",
      "ip": "192.168.1.100",
      "configurado": true,
      "conectadoDesde": "2026-08-31T15:20:00-04:00",
      "heartbeatEstado": "HEALTHY",
      "segundosInactivo": 45
    }
  ]
}
```

6. Características

El sistema incluye funcionalidades diseñadas para mejorar la robustez y la flexibilidad en entornos reales:

- Heartbeat automático: El POS envía una señal de "latido" cada 2 minutos cuando está en reposo. El servidor responde confirmando la conexión. Esto permite detectar y registrar desconexiones de forma temprana, y mantener actualizado el estado en /dispositivos.
- Soporte para DCC (Dynamic Currency Conversion): Si el POS y el adquirente lo soportan, el flujo de pago puede incluir la oferta de conversión de moneda. El servicio no requiere configuración adicional; cuando aplica, la respuesta de /pago/ trae source: "host_response" y datos adicionales del host de pago (ver ejemplo en 6.1).
- Módulo de propinas / donaciones: El sistema permite que, durante una venta con tarjeta, el usuario añada una propina o donación directamente en el POS. Este monto adicional se refleja en el campo additionalAmount de la respuesta. Para habilitar esta funcionalidad, el comercio debe

activar el flag useTips: true en el archivo index.js (ver sección 6.2). Si no se activa, el campo no aparecerá en la respuesta.

6.1 Ejemplo de respuesta con DCC

Cuando la transacción se procesa con conversión dinámica de moneda, la respuesta de /pago/ incluye información adicional del host de pago (source: "host_response") junto con los datos propios de la transacción:

```
{
  "status": "success",
  "source": "HOST",
  "useTips": true,
  "type": "host_response",
  "code": "77",
  "message": "VENTA DCC",
  "detail": "0000000",
  "identifier": "10060",
  "data": [
    { "name": "purchaseAmount", "value": "000000006000" },
    { "name": "tip", "value": "000000000000" },
    { "name": "receiptNumber", "value": "001339" },
    { "name": "RRN", "value": "624015001339" },
    { "name": "terminalID", "value": "00230801" },
    { "name": "transactionDate", "value": "0828" },
    { "name": "transactionTime", "value": "1557" },
    { "name": "responseCode", "value": "77" },
    { "name": "accountType", "value": "01" },
    { "name": "installmentNumber", "value": "00" },
    { "name": "last4Digits", "value": "7065" },
    { "name": "errorMessage", "value": "0000000" },
    { "name": "cardBINTarjeta", "value": "531084" }
  ]
}
```

6.2 Activación de propinas

En el archivo index.js, busque la línea donde se inicializa IntegrationPos y añada o modifique el parámetro useTips:

```
var NetServer = IntegrationPos.initialize({
  port: POS_PORT,
  host: "0.0.0.0",
  server: net.createServer(),
  devicesConfig: devicesConfig,
  captureDevice: (evt) => { ... },
  useTips: true   // <-- Cambiar a true para habilitar
});
```

Reinicie el servicio para que el cambio surta efecto.

7. Solución de problemas comunes

Problema	Posible causa	Solución
El servidor no arranca	devices.json no existe o está mal formado	Crear el archivo con un JSON válido (ver sección 3.3).
Device not conected	La IP del POS no coincide con la de devices.json	Verificar con GET /dispositivos la IP real de conexión y actualizar devices.json.
Dispositivo no configurado	La etiqueta enviada no existe en devices.json	Revisar ortografía y mayúsculas/minúsculas.
Error 409 (device_busy)	El POS ya está ejecutando otra operación	Esperar unos segundos y reintentar. Si persiste, reiniciar el servicio.
Timeout o falta de respuesta del POS	Problema de red o POS apagado	Verificar conectividad; aumentar POS_OPERATION_TIMEOUT_MS si es necesario.

8. Consideraciones finales

- El archivo index.js es modificable; puede personalizar rutas, añadir autenticación o lógica de negocio sin afectar la librería.
- Mantener la librería actualizada, de acuerdo a lo informado y lo solicitado, sea via .zip o mediante el repositorio. Los cambios estarán marcados en la rama master, debidamente identificados como nuevas versión o bugfixes, y se le informara siempre que la Version contenga Cambios importantes(Braking changes)
- Monitoree el estado de los POS a través del endpoint /dispositivos y los logs en el directorio logs/.

Este manual cubre los pasos necesarios para instalar, configurar y operar la API de integración con POS. Para soporte o consultas, contacte al equipo de desarrollo.